

Sclara 2.0

Architecture & Dapr Communication
A complete guide — browser to database and back

Spring Boot 2.6.5

Java 17

Dapr 1.12

Redis Pub/Sub

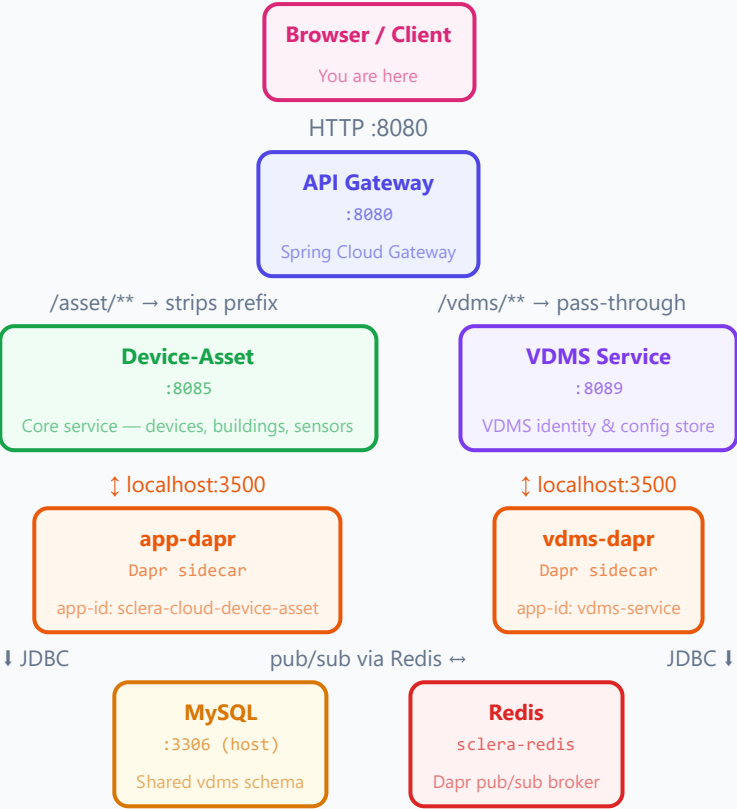
MySQL 8

Docker Compose

01 The Big Picture

Architecture

ALL CONTAINERS AT A GLANCE



One rule: The browser only ever talks to port **8080** (the gateway). Every other port is internal. Port **3000** is the test UI served by nginx (not part of the application stack).

02 What is Dapr?

Dapr

The short version

Dapr is a **middleman process** that sits next to your service and handles all the difficult networking problems so your code doesn't have to.

Think of it like a **post office clerk** sitting at your desk — you hand them a letter, they figure out the routing, retries, and delivery. You write business logic, not network plumbing.

What problems does it solve?

- 1

Service Discovery

You call `invoke/vdms-service/method/vdms/id` — Dapr finds where that service actually is.
- 2

Pub/Sub Messaging

Publish an event to a topic name. Dapr writes it to Redis, routes it to all subscribers. No direct connection needed.
- 3

Retries & Resiliency

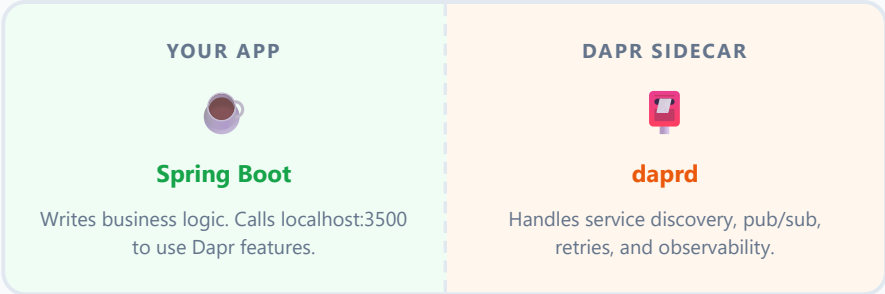
If a service is temporarily down, Dapr can retry. Your code makes one call and moves on.
- 4

Observability

Dapr logs every invoke and publish call, giving you a trace of exactly what called what.

The Sidecar Pattern

Each service gets its own Dapr process. They share the same network namespace — meaning they reach each other on **localhost**.



Both share the same Docker network namespace — they see each other on **localhost**

The two Dapr APIs used here

```
// 1. Synchronous call to another service GET
localhost:3500/v1.0/invoke/{app-id}/method/{path} // 2. Publish an async
event POST localhost:3500/v1.0/publish/{pubsub}/{topic}
```

Your app always talks to **localhost:3500**. Dapr handles everything else. No IP addresses, no service URLs, no load balancer config in code.

03 Services Overview

Architecture

SERVICE	PORT	APP-ID (DAPR)	TECH STACK	RESPONSIBILITY
API Gateway	8080	— (no Dapr)	Spring Cloud Gateway	Single entry point for all external HTTP. Routes <code>/asset/**</code> → device-asset, <code>/vdms/**</code> → vdms-service. No auth here.
Device-Asset	8085	sclera-cloud-device-asset	Spring Boot 2.6.5, Java 17, JDBC/JPA	Core business service. Manages devices, buildings, sensors, documents, AI suggestions. Does JWT validation. Calls vdms-service via Dapr. Publishes events to Redis.
VDMS Service	8089	vdms-service	Spring Boot, Java 17, JDBC/JPA	Lightweight identity & config store. Holds VDMS details, audit logs. Subscribes to events from device-asset. Can call back to device-asset via Dapr (2-way).
app-dapr	3500 (internal)	serves device-asset	daprio/daprd:1.12.0	Dapr sidecar for device-asset. Shares network namespace via <code>network_mode: service:app</code> .
vdms-dapr	3500 (internal)	serves vdms-service	daprio/daprd:1.12.0	Dapr sidecar for vdms-service. Shares network namespace via <code>network_mode: service:vdms-service</code> .
Redis	internal	—	redis:7-alpine	Message broker for Dapr pub/sub. Both sidecars connect here to exchange async events.
MySQL	3306 (host)	—	MySQL 8	Shared database. Both device-asset and vdms-service use the same <code>vdms</code> schema via <code>host.docker.internal:3306</code> .

Why two Dapr sidecars? Each service needs its own Dapr process because Dapr needs to know the **app-id** (service name) and **app-port** (where to deliver inbound calls) for the service it is paired with. One sidecar cannot serve two apps.

04

API Gateway — The Front Door

Architecture

What it does

The API Gateway is the **only container** that accepts traffic from the internet/browser. It is a pure reverse proxy — it strips or passes URL prefixes and forwards requests to the right backend service. It does **not** use Dapr, do authentication, or contain business logic.

Routing Rules

INCOMING PATH	FORWARDS TO	TRANSFORM
/asset/**	device-asset:8085	Strips /asset prefix
/vdms/**	vdms-service:8089	Pass-through (no change)

Example: Browser calls GET
/asset/user/admin/vdms/V1/docker/d1/devices
Gateway strips /asset → forwards GET
/user/admin/vdms/V1/docker/d1/devices to device-asset:8085

Request path through the gateway

- 1

Browser sends request

GET http://localhost:8080/asset/user/{u}/vdms/{v}/docker/{d}/devices
- 2

Gateway matches route /asset/**

Strips the /asset prefix from the URL.
- 3

Gateway forwards to device-asset

GET http://sclera-cloud-device-asset:8085/user/{u}/vdms/{v}/docker/{d}/devices
- 4

Response passes back through gateway

The gateway streams the response back to the browser. Status code, headers, and body are unchanged.

05

Device-Asset Service

Architecture

What it manages

This is the core of the application. Every device, building, floor, sensor, document, and AI suggestion lives here. It is where all CRUD (Create, Read, Update, Delete) operations happen.

Key controllers (endpoints)

CONTROLLER	HANDLES
DeviceController	Add / edit / delete / list / search / sort devices
BuildingController	Buildings & locations management
SensorController	Sensor data & configuration
DocumentController	File attachments and documents

URL pattern for all device endpoints

/user/{username}/vdms/{vdmsId}/docker/{dockerName}/...

Every endpoint is scoped to a specific VDMS installation identified by **username**, **vdmsId**, and **dockerName**.

How it talks to VDMS Service

```
// VdmsClient.java (device-asset side) public Map getVdmsDetails() { //
calls localhost:3500 (its own Dapr sidecar) return invoke("vdms/details",
Map.class); } private T invoke(String path, Class<T> type) { String url =
daprBase + "/v1.0/invoke/vdms-service/method/" + path; return
rest.getForEntity(url, type).getBody(); }
```

How it publishes async events

```
// UserActionLogService.java triggers this
vdmsClient.publishEvent("device.audit", payload); //
VdmsClient.publishEvent() does: POST
localhost:3500/v1.0/publish/pubsub/device.audit // Dapr writes to Redis →
delivered to vdms-service
```

Publishing is **fire-and-forget**. The service returns to the caller immediately. Dapr + Redis guarantee the message reaches the subscriber — even if vdms-service is momentarily down.

06 VDMS Service

What it stores

VDMS stands for **Virtual Device Management System**. This service is a dedicated store for VDMS identity and configuration — things like the property address, customer org ID, whether a VDMS is a master unit, and all device audit logs.

Controllers

CLASS	ROLE
VdmsController	Serves VDMS identity data (id, details, master, customer org). Also registers Dapr subscriptions via <code>GET /dapr/subscribe</code> .
VdmsSubscriber	Receives CloudEvents from device-asset. Updates VDMS table rows when property details or org ID change.
AuditSubscriber	Receives <code>device.audit</code> CloudEvents. Persists them to <code>user_action_log</code> table. Also exposes <code>GET /vdms/audit-log</code> for clients to read them back.
AssetProxyController NEW	2-way proxy. Calls back to device-asset via its own Dapr sidecar. Endpoints: <code>/vdms/asset/{u}/{v}/{d}/device-count</code> and <code>/devices</code> .

How Dapr delivers events to VDMS

When vdms-service starts, Dapr calls its `GET /dapr/subscribe` endpoint to find out which topics it listens to. Dapr then routes matching Redis messages to the registered POST handlers.

```
// VdmsController returns this list to Dapr [ { "pubsubname": "pubsub",
"topic": "vdms.update-property-details", "route": "/vdms/update-property-
details" }, { "topic": "device.audit", "route": "/vdms/device-audit" }, //
... more topics ]
```

New: Calling back to Device-Asset

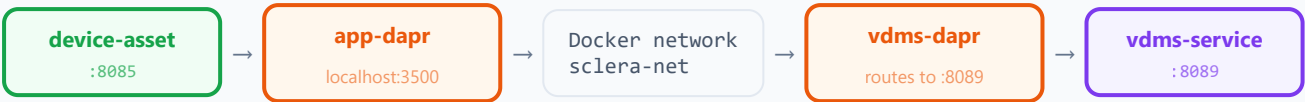
```
// ScleraCloudDeviceClient.java (vdms-service) private static final String
APP_ID = "sclera-cloud-device-asset"; public Object getDevices(String u,
String v, String d) { String path =
"user/"+u+"/vdms/"+v+"/docker/"+d+"/devices"; // calls its OWN sidecar on
localhost:3500 return invoke(path, Object.class); }
```

Both services use the exact same Dapr invoke pattern. The only difference is the **app-id** they target.

07 Communication Channel 1 — Synchronous Invoke

device-asset → vdms-service (synchronous query)

Used when device-asset needs VDMS data *right now* to process a request. The caller waits for the response.



URL pattern

localhost:3500/v1.0/invoke/vdms-service/method/{path}

All VdmsClient methods

JAVA METHOD	MAPS TO VDMS-SERVICE ENDPOINT	RETURNS
getVdmsId()	GET /vdms/id	{"vdmsId": "..."}
getVdmsDetails()	GET /vdms/details	Full VdmsDTO
getMaster()	GET /vdms/master	{"isMaster": 0 1}
getHasSecondaryDevice()	GET /vdms/has-secondary-device	{"hasSecondaryDevice": 0 1}
getSecondaryDeviceId()	GET /vdms/secondary-device-id	{"secondaryDeviceId": "..."}
getCustomerOrgId(vdmsId)	GET /vdms/customer-org-id/{vdmsId}	{"customerOrgId": "..."}
getSyncDetailsForAdc()	GET /vdms/sync-details-for-adc	Subset of VdmsDTO

Step-by-step trace

- 1

Service calls VdmsClient

e.g., `vdmsClient.getVdmsDetails()` inside a service method
- 2

VdmsClient calls its own Dapr sidecar

GET localhost:3500/v1.0/invoke/vdms-service/method/vdms/details
- 3

app-dapr resolves "vdms-service"

Dapr looks up the container with app-id `vdms-service` and forwards to its sidecar (vdms-dapr) over the Docker bridge network.
- 4

vdms-dapr delivers to Spring Boot

GET http://localhost:8089/vdms/details
- 5

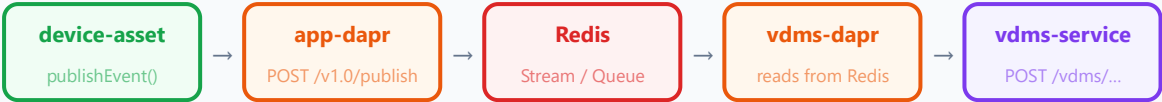
Response flows back

vdms-service → vdms-dapr → network → app-dapr → VdmsClient → caller. Synchronous, returns the JSON map.

08 Communication Channel 2 — Pub/Sub (Async Events)

Dapr

device-asset → Redis → vdms-service (fire-and-forget)



All pub/sub topics

TOPIC	PUBLISHED WHEN	HANDLED BY	EFFECT
vdms.update-property-details	Property address changes	VdmsSubscriber	Updates address column in VDMS table
vdms.update-customer-org-id	Org ID changes	VdmsSubscriber	Updates customer_org_id column
vdms.set-agent-permission	Agent permission change	VdmsSubscriber	Logs event (handler extensible)
device.audit	Device add / edit / delete	AuditSubscriber	Persists row to user_action_log

What a CloudEvent looks like

Dapr wraps your payload in a CloudEvent envelope before delivering it to the subscriber.

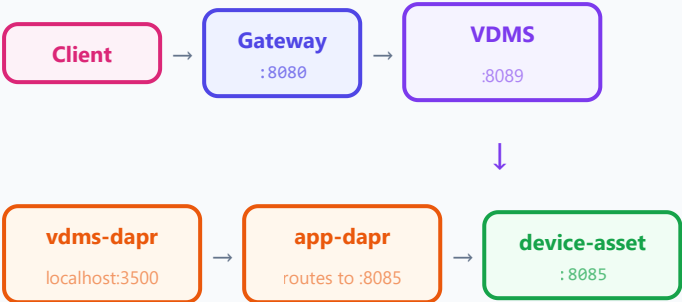
```
{ "specversion": "1.0", "type": "com.dapr.event.sent", "source": "sclera-cloud-device-asset", "topic": "device.audit", "pubsubname": "pubsub", "data": { "vdmsId": "VDMS760", "action": "ADD", "deviceId": "dev-123", "userEmail": "admin@sclera.com", "message": "Device 'Lobby Sensor' added" } }
```

The subscriber reads the **data** field. The outer envelope is boilerplate added/removed by Dapr automatically.

09 Communication Channel 3 — 2-Way (Reverse Call)

vdms-service → device-asset

The new reverse direction. vdms-service can now call back to device-asset — useful when a client queries `/vdms/asset/...` and expects device data sourced from device-asset, routed through vdms-service.



New endpoints in vdms-service

ENDPOINT	PROXIES TO
GET /vdms/asset/{u}/{v}/{d}/device-count	GET /user/{u}/vdms/{v}/docker/{d}/getdevicecount on device-asset
GET /vdms/asset/{u}/{v}/{d}/devices	GET /user/{u}/vdms/{v}/docker/{d}/devices on device-asset

ScleraCloudDeviceClient.java

```
// In: sclera-vdms-service/.../client/ @Service public class
ScleraCloudDeviceClient { private static final String APP_ID =
"sclera-cloud-device-asset"; public Object getDevices(String u,
String v, String d) { return invoke(
"user/"+u+"/vdms/"+v+"/docker/"+d+"/devices", Object.class ); }
private T invoke(String path, Class<T> type) { String url =
daprBase + "/v1.0/invoke/" + APP_ID + "/method/" + path; return
rest.getForEntity(url, type).getBody(); } }
```

AssetProxyController.java

```
@RestController @RequestMapping("/vdms/asset") public class
AssetProxyController { @GetMapping("/{u}/{v}/{d}/devices") public
ResponseEntity<Object> getDevices( @PathVariable String u, v, d)
{ Object result = assetClient.getDevices(u, v, d); if (result ==
null) return ResponseEntity.status(502)...; return
ResponseEntity.ok(result); } }
```

10 Request Flow — Device List Query

Pin-to-pin trace

1 Browser / Test UI sends request

```
GET
http://localhost:8080/asset/user/admin/vdms/VDMS760/docker/right_wing/devices
```

With header: `Authorization: Bearer <JWT>`

2 API Gateway — route match

Matches `/asset/**`. Strips the prefix and forwards.

```
GET http://sclera-cloud-device-asset:8085/user/admin/vdms/VDMS760/docker/right_wing/devices
```

3 Device-Asset — JWT Validation

JWT filter validates the Bearer token. Multi-tenant: it reads the issuer claim to pick the right JWK set. Rejects 401 if invalid.

4 DeviceController → DeviceService

Controller extracts path variables, calls DeviceService. If VDMS data is needed (e.g., permission check), it calls VdmsClient.

5 (optional) VdmsClient → Dapr → vdms-service

```
GET localhost:3500/v1.0/invoke/vdms-service/method/vdms/details
```

6 DeviceService queries MySQL

JDBC query against the `vdms` schema returns the device list as a `Set<DeviceDTO>`.

7 Response returns to browser

JSON array of devices flows back through gateway to browser. HTTP 200.

As a diagram

```
Browser → :8080 → Gateway strips /asset →
device-asset:8085 JWT check →
DeviceController → DeviceService
↳ (if needed) VdmsClient → localhost:3500
→ Dapr → vdms-service:8089 → MySQL
↳ MySQL → Set<DeviceDTO>
← ← ← JSON array ← ← ← Browser
```

Key facts about this flow

- Auth happens inside device-asset**

The gateway passes the JWT header through. JWT validation is done by the `WebSecurityConfig` filter chain in device-asset, not the gateway.
- VdmsClient call is optional**

Device-asset only calls vdms-service when it actually needs VDMS data. Many device endpoints work purely from MySQL without touching the VDMS service.
- Shared MySQL schema**

Both services connect to the same MySQL `vdms` database. They each own their own tables within that schema.

11 Request Flow — Device Add + Audit Log

End-to-End Flow

Pin-to-pin trace

- 1 Browser POSTs a new device**

```
POST http://localhost:8080/asset/user/admin/vdms/VDMS760/docker/right_wing/adddevice
```


Body: {"user_data_name":"Lobby Sensor","type":"generic",...}
- 2 Gateway strips prefix → device-asset**

```
POST http://sclera-cloud-device-asset:8085/user/admin/vdms/VDMS760/docker/right_wing/adddevice
```
- 3 DeviceController → DeviceService.addDevice()**
JWT validated. DeviceService builds the DeviceDTO and writes it to MySQL via JDBC.
- 4 MySQL write completes**
Device row inserted into the `device` table in the `vdms` schema.
- 5 UserActionLogService publishes audit event**

```
POST localhost:3500/v1.0/publish/pubsub/device.audit
```


Payload: vdmsId, action="ADD", deviceId, userEmail, timestamp. app-dapr accepts it and writes to Redis immediately.
- 6 HTTP 200 back to browser**
Device-asset returns the created device. The audit event delivery to vdms-service is happening **asynchronously in the background** — the user doesn't wait for it.
- 7 Redis → vdms-dapr → AuditSubscriber**

```
POST http://localhost:8089/vdms/device-audit (CloudEvent)
```


vdms-service's Dapr sidecar reads from Redis and delivers the CloudEvent. AuditSubscriber unwraps the data and inserts a row into `user_action_log`.

Timeline — what happens when

```
t=0ms  Browser POST
t=5ms  Gateway forwards
t=10ms  JWT validated
t=15ms  MySQL INSERT done
t=18ms  Dapr publish accepted
t=20ms  ← HTTP 200 to browser

--- async, after response ---
t=25ms  Redis delivery
t=30ms  vdms-service: audit row written
```

Why async for audit?

Speed: The user gets an instant response. Audit logging is a side-effect — there's no reason to make the user wait for it.

Reliability: Redis + Dapr guarantees at-least-once delivery. Even if vdms-service crashes at step 7, the audit event stays in Redis until it's successfully processed.

Eventual consistency: The audit log entry appears a few milliseconds after the device is created. If you query the audit log immediately after adding a device, it might not appear yet — refresh in a second.

12

Request Flow — Audit Log Query

End-to-End Flow

Pin-to-pin trace

- 1 Browser requests audit log**
`GET http://localhost:8080/vdms/audit-log?vdmsId=VDMS760&page=0&size=20`
- 2 Gateway matches `/vdms/**`**
Pass-through (no prefix stripping). Forwards to vdms-service.
`GET http://vdms-service:8089/vdms/audit-log?vdmsId=VDMS760&page=0&size=20`
- 3 `AuditSubscriber.getAuditLog()`**
Controller reads query params: vdmsId, page (0-based), size (capped at 100).
- 4 `UserActionLogRepository` queries MySQL**
`findByVdmsIdOrderByCreatedAtDesc(vdmsId, pageable)` — returns newest entries first, paginated.
- 5 JSON array returned to browser**
Each entry: id, action, deviceId, userEmail, message, createdAt. HTTP 200.

Closing the loop

`device-asset` adds device → MySQL
`device-asset` publishes `device.audit` event
`Redis` stores event
`vdms-service` consumes event → MySQL

`Client` queries `vdms-service`
`vdms-service` reads MySQL → returns log

This is the **write-once, read-anywhere** pattern. `device-asset` writes events; any client can later read them from `vdms-service` without `device-asset` being involved.

Also: 2-way query flow (new)

- 1 Browser queries via VDMS route**
`GET /vdms/asset/admin/VDMS760/right_wing/devices`
- 2 Gateway → vdms-service (pass-through)**
`/vdms/**` routes to `vdms-service:8089`
- 3 `AssetProxyController` calls `SclaraCloudDeviceClient`**
`GET localhost:3500/v1.0/invoke/sclara-cloud-device-asset/method/user/.../devices`
- 4 `vdms-dapr` → `device-asset:8085` → MySQL**
Device list flows back through the Dapr invoke chain to the browser.

Container startup order

- 1

redis (sclera-redis)

Starts first. Health-checked with `redis-cli ping`. All other containers wait for it to be healthy.
- 2

app (sclera-cloud-device-asset)

Starts after redis. Spring Boot on :8085. Connects to `host.docker.internal:3306` for MySQL (MySQL runs on the host machine, not in Docker).
- 3

vdms-service

Same as app — starts after redis, connects to host MySQL on :3306.
- 4

app-dapr & vdms-dapr (sidecars)

Start after their paired service. Use `network_mode: "service:app" / "service:vdms-service"` — they share the network namespace of the service container.
- 5

api-gateway

Starts last, after both services are up. Exposes :8080 externally.
- 6

ui (sclera-test-ui)

nginx serving `./test-ui/` on :3000. `demo.html`, `architecture.html`, etc. are served from here.

The `network_mode: service:X` trick

This is the key to how Dapr sidecars work in Docker Compose. When a container uses `network_mode: "service:app"`, it:

- ✓

Shares the same network namespace

The sidecar has the exact same IP address and ports as the main service container. `localhost` inside device-asset reaches app-dapr on port 3500.
- ✓

No separate IP needed

The sidecar has no dedicated port mapping. Communication between app and sidecar is purely through `localhost`.
- !

Rebuild caveat

If you rebuild the service container (`docker-compose up --build`), the old container is destroyed. The sidecar's `network_mode` now points to a dead container — **you must recreate the sidecar too:**

```
docker-compose stop vdms-dapr && docker-compose rm -f vdms-dapr && docker-compose up -d vdms-dapr
```

Port map at a glance

CONTAINER	HOST PORT	CONTAINER PORT
sclera-api-gateway	8080	8080
sclera-cloud-device-asset	8085	8085
vdms-service	8089	8089
sclera-test-ui (nginx)	3000	80
MySQL (host process)	3306	—
sclera-redis	internal only	—
app-dapr / vdms-dapr	internal only	3500

14

Quick Reference Card

Reference

Dapr URL patterns (memorize these two)

— Synchronous service call — GET localhost:3500 /v1.0/invoke/{app-id}/method/{path} — Async publish — POST localhost:3500 /v1.0/publish/{pubsub-name}/{topic} Content-Type: application/json Body: your JSON payload

App IDs

SERVICE	DAPR APP-ID
Device-Asset	sclera-cloud-device-asset
VDMS Service	vdms-service
Pub/Sub component name	pubsub

Common pub/sub topics

TOPIC	DIRECTION
device.audit	device-asset → vdms
vdms.update-property-details	device-asset → vdms
vdms.update-customer-org-id	device-asset → vdms
vdms.set-agent-permission	device-asset → vdms

Key client classes

CLASS	LIVES IN	CALLS TO
VdmsClient	device-asset	vdms-service via app-dapr
ScleraCloudDeviceClient	vdms-service	device-asset via vdms-dapr

Rebuild checklist

1. Build the JAR first mvn -B clean package -DskipTests # 2. Rebuild & restart the service docker-compose up --build -d vdms-service # 3. Recreate the Dapr sidecar (CRITICAL) docker-compose stop vdms-dapr docker-compose rm -f vdms-dapr docker-compose up -d vdms-dapr

Test UI endpoints

URL	WHAT IT IS
localhost:3000/demo.html	Interactive test UI — all flows
localhost:3000/architecture.html	This document
localhost:8080/vdms/id	Quick health check
localhost:8080/vdms/details	VDMS config data
localhost:8080/vdms/audit-log?vdmsId=...	Audit log (paginated)

3 COMMUNICATION CHANNELS

- 1 **Sync Invoke**
device-asset → VdmsClient → Dapr → vdms-service
Real-time query. Caller waits for response.
- 2 **Pub/Sub (Async)**
device-asset → Dapr → Redis → vdms-service
Fire-and-forget. Used for audit, state changes.
- 3 **Reverse Invoke (2-way)**
vdms-service → SclaraCloudDeviceClient → Dapr → device-asset
New. vdms-service can proxy device data back to clients.

5 KEY DESIGN DECISIONS

- **Single gateway, port 8080**
No service is exposed directly. All traffic enters through the gateway.
- **JWT auth inside device-asset**
Not at the gateway. Multi-tenant: each issuer has its own JWK set.
- **Dapr for all inter-service calls**
No hardcoded service URLs. Dapr resolves by app-id. Easy to move services.
- **Async audit via pub/sub**
Audit logging never slows down the user's request. Redis persists events if vdms-service is down.
- **Shared MySQL schema**
Both services use the same DB. Simple for now — could be split if services need to scale independently.

THE COMPLETE MESSAGE PATH

```

Browser
  ↓ HTTP :8080
API Gateway (route + strip)
  ↓ HTTP :8085
device-asset (JWT + logic)
  ↓ localhost:3500
app-dapr (service invoke)
  ↓ Docker network
vdms-dapr → vdms-service
  ↓ MySQL query
  ↑ response back
app-dapr (pub/sub publish)
  ↓ localhost:6379
Redis → vdms-dapr
  ↓ CloudEvent POST
vdms-service (subscriber)
  ↓ MySQL INSERT
user_action_log
  
```

The loop is complete. A device add by a user in the browser ends up as a device row in MySQL AND an audit entry in MySQL — via two separate paths, one synchronous and one async.